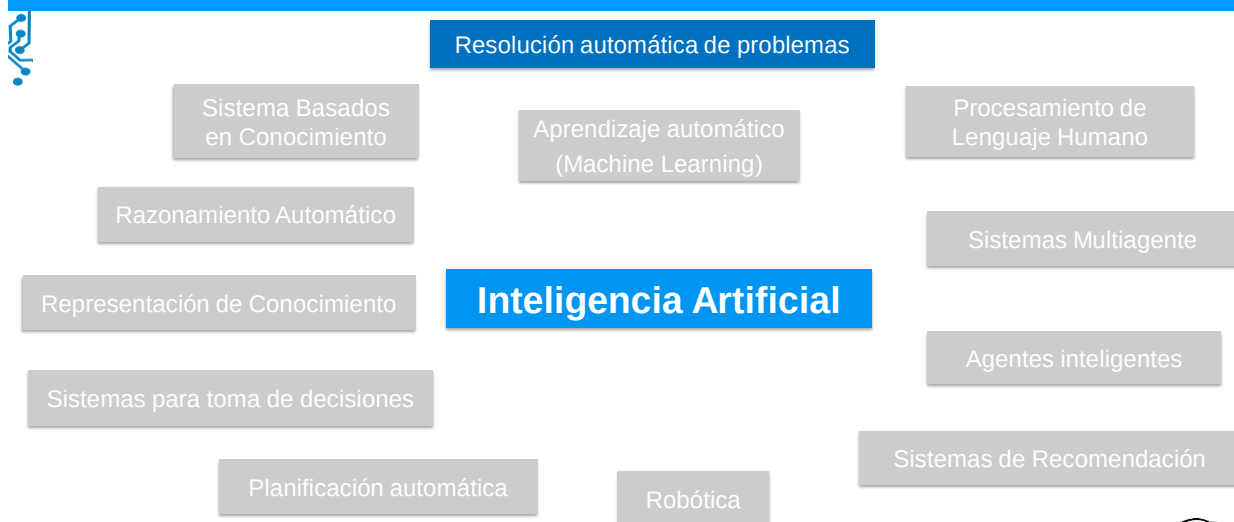




(Algunas) áreas de Inteligencia Artificial



"Es la Ciencia e Ingeniería de construir artefactos inteligentes, en especial programas inteligentes de computadora. También se relaciona con el uso de computadoras para entender la inteligencia humana".



Capacidad para resolver problemas



La **inteligencia** está muy relacionada con la capacidad de resolver problemas [RAE].

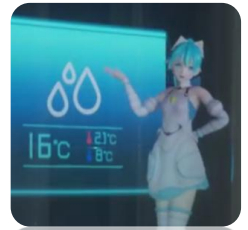
Bajo determinados enfoques, podríamos decir que un agente es inteligente si tiene la capacidad de resolver problemas. Cuantos más problemas diferentes pueda resolver, sería más inteligente.

Volviendo sobre dos ejemplos del episodio anterior.

Un **asistente virtual** sería inteligente si se adapta fácilmente y nos resuelve toda clase de problemas (o al menos nos ayuda a resolver una gran variedad de problemas).

El **robot de delivery**, aunque al principio pueda parecer muy enfocado en un dominio particular, demostraría cierta inteligencia si es capaz de resolver toda clase de adversidades en su meta por entregar un pedido (pensar esto me hizo acordar a **WALL-E**).

Aunque no aceptemos a “la capacidad de resolver problemas” como la definición definitiva para **inteligencia**, no deberíamos negar que es una capacidad muy importante y que no deberíamos subestimarla al momento de construir agentes inteligentes.



IA - Alejandro J. García

Introducción



Es interesante observar, que los humanos cuando tenemos un problema que resolver, en general lo que decimos es: “**vamos a buscar una solución**”.

La **resolución automática de problemas usando búsqueda** es central en IA.

¿Por qué “**búsqueda**”?

Porque para una gran variedad de situaciones, la resolución de un problema se puede **transformar** en la tarea de **encontrar un camino** en un grafo.

¿Por qué se le da tanta importancia?

Porque para una amplia clase de problemas se puede tener un **método** de resolución de problemas **genérico** que sea independiente de un dominio en particular.

¿A cuáles problemas se puede aplicar esta idea?

A todos aquellos que podamos definir como un “**problema de búsqueda**”.



IA - Alejandro J. García

Definición de problema de búsqueda

Problema ejemplo: Puzzle de Cajas



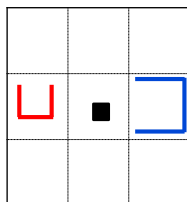
Para definir “problema de búsqueda” te propongo reflexionar (y también jugar) a este pasatiempo.

Problema ejemplo: Puzzle de Cajas

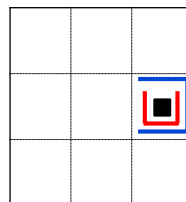
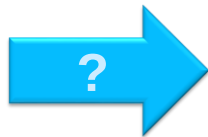
Instrucciones:

El “cursor” (cuadrado negro) permite mover las cajas por la grilla empujando desde adentro.

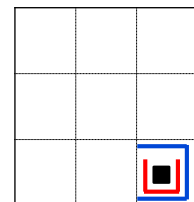
El juego (desafío) termina si se logra dejar la “caja” roja dentro de la caja “azul”.



inicio



Una posible meta



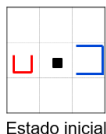
Otra posible meta

Podes jugarlo acá: [Boxup Puzzle Game \(mathsisfun.com\)](https://mathsisfun.com/Boxup-Puzzle-Game/)

ADVERTENCIA: es adictivo. Jugar en exceso no es bueno.



Terminología y Conceptos Básicos



EJEMPLO

Un problema de búsqueda se define indicando:

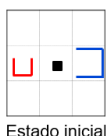
1. La descripción del **estado inicial** desde donde comenzar la búsqueda.
- 2.
- 3.
- 4.

voy a usar como
ejemplo al puzzle
de cajas



IA - Alejandro J. García

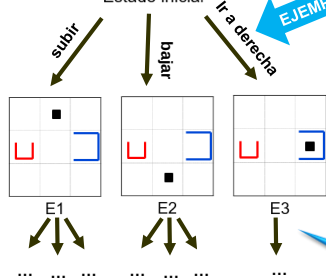
Terminología y Conceptos Básicos



EJEMPLOS

Un problema de búsqueda se define indicando:

1. La descripción del **estado inicial** desde donde comenzar la búsqueda.
2. La descripción de los **operadores disponibles** con los cuales el agente puede transformar un estado en otro.
- 3.
- 4.



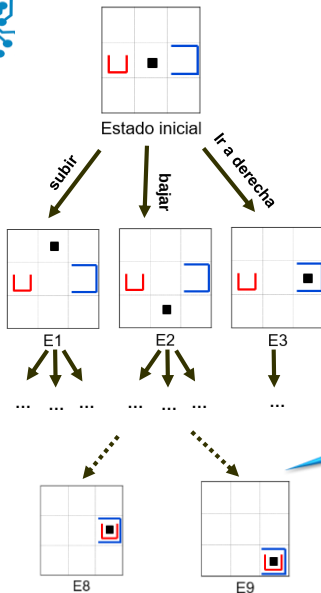
En este ejemplo hay cuatro operadores disponibles:
subir cursor, bajar cursor, cursor a derecha, cursor a izquierda.

Se llama **espacio de estados** (*state space*) o también **espacio de búsqueda** (*search space*) al conjunto de todos los estados alcanzables a partir del estado inicial mediante los operadores disponibles.



IA - Alejandro J. García

Terminología y Conceptos Básicos



Un problema de búsqueda se define indicando:

1. La descripción del **estado inicial** desde donde comenzar la búsqueda.
2. La descripción de los **operadores disponibles** con los cuales el agente puede transformar un estado en otro.
3. Un **test de meta** que determina si un estado es una meta.
- 4.

Si el problema tiene solución, entonces algunos estados del espacio de búsqueda serán un estado meta.

En este ejemplo: un estado es meta si la caja roja está en la misma celda que al caja azul.

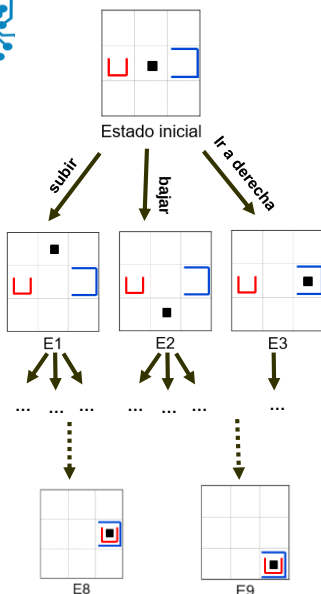
meta(E1) = false
meta(E3) = false

meta(E8) = true
meta(E9) = true



IA - Alejandro J. García

Terminología y Conceptos Básicos



Un problema de búsqueda se define indicando:

1. La descripción del **estado inicial** desde donde comenzar la búsqueda.
2. La descripción de los **operadores disponibles** con los cuales el agente puede transformar un estado en otro.
3. Un **test de meta** que determina si un estado es una meta.
4. Una **función de costo de camino** $g(n)$ que determina el costo para llegar del estado inicial a otro estado "n".

Una solución a un problema de búsqueda es un camino desde el estado inicial a una meta (puede no ser única).

Una solución es óptima si tiene **menor costo** de camino que todas las demás soluciones.

meta(E1) = false
meta(E3) = false

$g(E1) = 1$
 $g(E3) = 1$

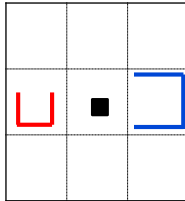
meta(E8) = true
meta(E9) = true

$g(E8) = 5$
 $g(E9) = 9$



IA - Alejandro J. García

Ejemplo : Puzzle de Cajas



Estado inicial

Ejemplo de problema de búsqueda para el puzzle de cajas

1. Estado inicial: (ver dibujo).
2. Operadores (son 4):
subir cursor, bajar cursor, cursor a derecha, cursor a izquierda.
3. Test de meta: caja roja y azul en la misma celda.
4. Costo de cada operador = 1 entonces $g(N) = \text{profundidad}(N)$

¿Representación de conocimiento?

Una opción es representar cada estado con una lista de predicados que establece la posición de cada elemento. A continuación hay dos ejemplos en notación de Prolog:

estado = [roja(2,1,arriba), cursor(2,2), azul(3,2,izquierda)]

estado = [roja(3,2,arriba), cursor(3,2), azul(3,2,izquierda)]

Es test de meta entonces podría definirse en Prolog de esta manera:

Meta(E) :- member(roja(X,Y,_),E), member(azul(X,Y,_),E).



IA - Alejandro J. García

Ejemplo: "15-Puzzle" o 15-tejas



Estado Inicial



Meta

Ejemplo de problema de búsqueda para las 15 tejas

1. Estado inicial: ??????????????
2. Operadores: ??????????????
3. Test de meta: ??????????????
4. Costo de cada operador ??????????????

Hay que tener en cuenta que cuanto más operadores, se aumenta el factor de ramificación en el espacio de búsqueda.



IA - Alejandro J. García

Ejemplo: "15-Puzzle" o 15-tejas



Estado Inicial



Meta

Ejemplo de problema de búsqueda para las 15 tejas

1. Estado inicial: (ver figura).
2. Operadores (son 4):
subir hueco, bajar hueco, hueco a derecha, hueco a izquierda.
3. Test de meta: las tejas ordenadas de menor a mayor (ver figura).
4. Costo de cada operador = 1 entonces $g(N) = \text{profundidad}(N)$



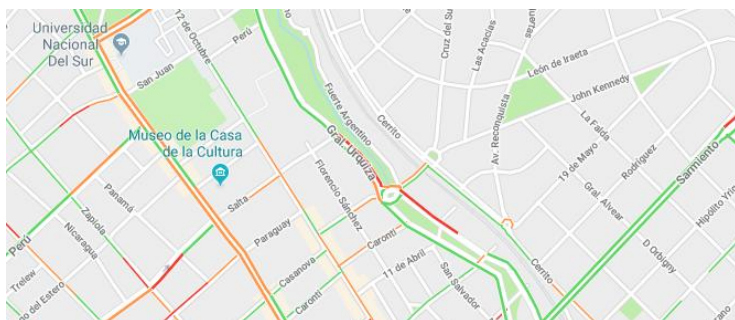
IA - Alejandro J. García

Agente delivery



Ejemplo de problema de búsqueda para un agente de delivery simple

1. Estado inicial: ????????????????
2. Operadores: ??????????????????
3. Test de meta: ??????????????????
4. Costo de cada operador con la función ??????????????????



Hay que tener en cuenta que cuanto más operadores, se aumenta el factor de ramificación en el espacio de búsqueda.

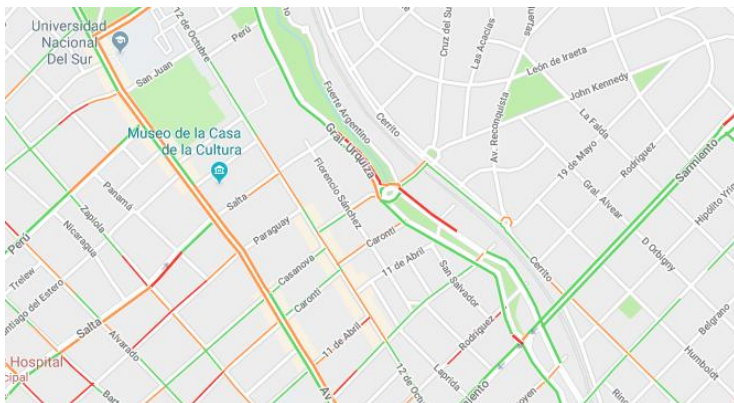


IA - Alejandro J. García

Agente delivery

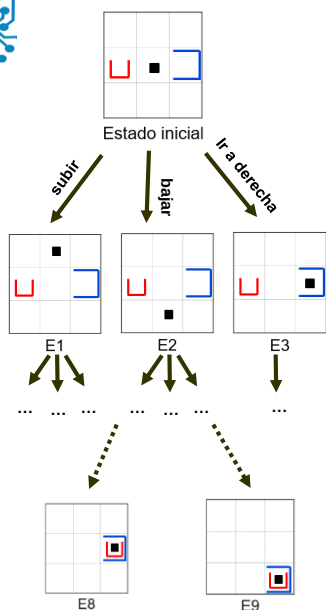
Ejemplo de problema de búsqueda para un agente de delivery simple

1. Estado inicial: en depósito y pedido para entregar.
2. Operadores: avanzar, doblar, cargar pedido, dejar pedido, saludar, esperar propina, calcular trayectoria.
3. Test de meta: pedido entregado y robot en el depósito.
4. Costo de cada operador con la función costo: operadores → Entero



IA - Alejandro J. García

Terminología y Conceptos Básicos



Estado: representación finita del mundo para un momento dado, expresada en algún formalismo.

Estado inicial: el estado desde donde se comienza a resolver el problema.

Operador: es una función que si se puede aplicar en un estado, lo transforma en otro estado (también se lo suele llamar *regla*, *transición*, *función sucesor*, *producción*, o *acción*).

Estado alcanzable: es un estado al cual se puede llegar desde el estado inicial aplicando una secuencia de operadores.

Espacio de búsqueda o espacio de estados: es el conjunto de todos los estados alcanzables desde el estado inicial.

IA - Alejandro J. García

Terminología y Conceptos Básicos



El **espacio de estados** se puede representar por un **grafo** donde cada **nodo** representan un **estado** de dicho espacio, y cada **arco** representa la aplicación de un **operador del problema de búsqueda**.

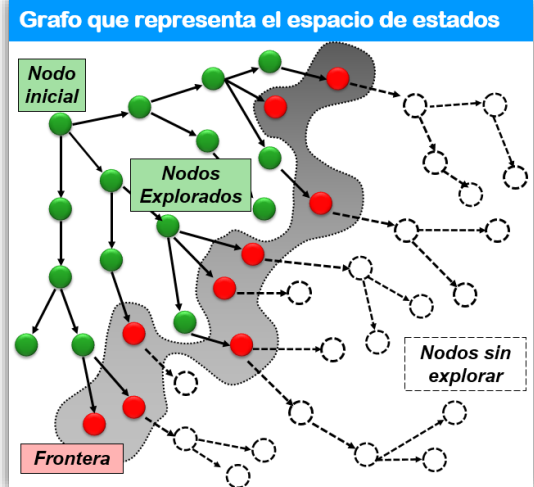
Al grafo que representa el **espacio de estados** también se llama "**espacio de búsqueda**" o "**grafo de búsqueda**".

El **grafo de búsqueda** se puede generar de manera **dinámica** a medida que la búsqueda avanza.

La **frontera** es el límite entre lo que ya fue explorado y lo que queda por explorar.

El **factor de ramificación** está determinado por el número de operadores aplicables en cada estado.

El número de estados posibles podría ser infinito.



IA - Alejandro J. García

Terminología y Conceptos Básicos



Al grafo que representa el **espacio de estados** también puede ser un **árbol** que tiene como raíz el estado inicial y se suele llamar "**árbol de búsqueda**".

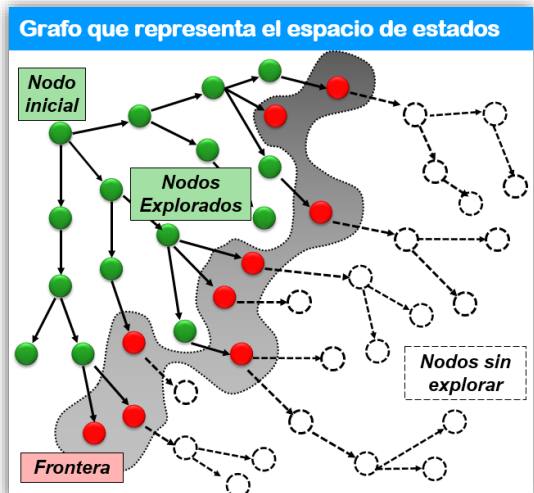
El **nivel de profundidad** de un nodo es el número de arcos desde el estado inicial hasta dicho nodo.

Estado meta (goal): el estado que satisface el test de meta (puede no ser único).

Meta Alcanzable: es una meta que es un estado alcanzable.

Costo de paso (step cost) es una función que asigna un costo a la aplicación de los operadores.

Costo de camino (path cost) $g(N)$: determina el costo para llegar del estado inicial a un nodo " N ", calculado en base a los costos de cada paso (arcos).



Observá la figura. ¿Cómo sería la representación computacional del árbol?
¿Dónde debo mirar si hay una meta? ¿En que lugar voy a encontrar una meta?

IA - Alejandro J. García

Métodos de búsqueda para resolución automática de problemas

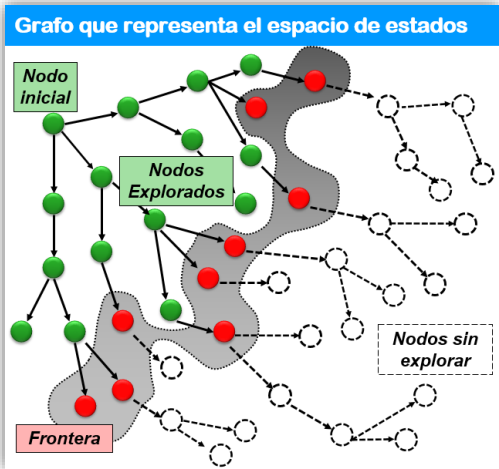
Algoritmo de búsqueda genérico



Buscar una meta en la frontera F del grafo de estado

Caso base:
cuando **elijo** un nodo N de F
y N satisface el test de meta

Caso recursivo:
cuando **elijo** un nodo N de F y N no es una meta,
entonces, primero sacar a N de F, luego **agregar** a la
frontera todos los nodos a los que puedo llegar con
algún operador aplicable a N (podría no haber ninguno),
y finalmente **buscar una meta en la nueva frontera**.



Elijo	Agregar	Estrategia
El primero de "Frontera"	Al principio de Frontera	búsqueda en profundidad
El primero de "Frontera"	Al final de Frontera	búsqueda a lo ancho



Algoritmo de búsqueda genérico



Buscar una meta en la frontera F del grafo de estado

Caso base:

cuando **elijo** un nodo N de F
y N satisface el test de meta

Caso recursivo:

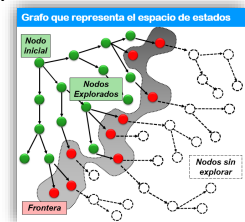
cuando **elijo** un nodo N de F y N no es una meta,
entonces, primero sacar a N de F , luego **agregar** a la
frontera todos los nodos a los que puedo llegar con
algún operador aplicable a N (podría no haber ninguno),
y finalmente **buscar una meta en la nueva frontera**.

buscarEn(Frontera) :- % recibe una lista de nodos

seleccionar(Nodo, Frontera, _), %selecciona 1 nodo
esMeta(Nodo). % si es meta da éxito

buscarEn(Frontera) :-

seleccionar(Nodo, Frontera, **FronteraReducida**),
generarVecinos(Nodo, **Vecinos**),
agregar(**FronteraReducida**, **Vecinos**, **NuevaFrontera**),
buscarEn(**NuevaFrontera**).



IA - Alejandro J. García

Algoritmo de búsqueda genérico



% caso base (encuentra una meta)

buscarEn(Frontera) :- % recibe una lista de nodos

seleccionar(Nodo, Frontera, _), %selecciona 1 nodo
esMeta(Nodo). % si es meta da éxito

% caso recursivo (amplia la frontera y sigue)

buscarEn(Frontera) :-

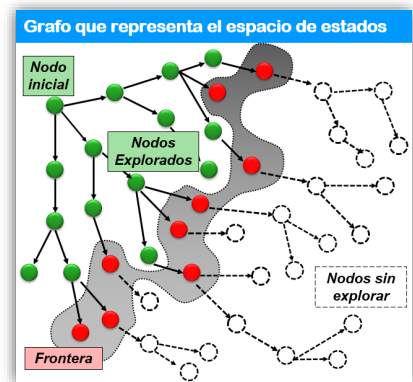
seleccionar(Nodo, Frontera, **FronteraReducida**),
generarVecinos(Nodo, **Vecinos**),
agregar(**FronteraReducida**, **Vecinos**, **NuevaFrontera**),
buscarEn(**NuevaFrontera**).

% selecciona un nodo de la frontera, y lo saca

% genera sus vecinos,

% agrega los nuevos a la frontera y

% continua la búsqueda con la nueva frontera.



IA - Alejandro J. García

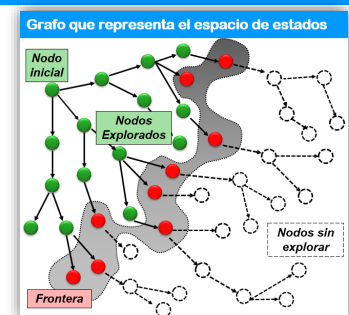
Búsqueda a ciegas

Resolución de Problemas utilizando búsqueda



Métodos de búsqueda no informados (“ciegos”)

- búsqueda en profundidad (DFS)
- búsqueda a la ancho (BFS)
- Low cost-first (LCFS)
también llamado Uniform-cost Search (UCS)



Cada uno de los métodos detallados sigue una estrategia diferente de búsqueda, pero **todos recorren el espacio de búsqueda hasta que se “tropiezan” con la solución.**

Exploran el espacio de búsqueda “a ciegas” **sin usar ningún tipo de información** del problema que se está resolviendo.



Estrategia de búsqueda

La forma de **seleccionar** el nodo a **expandir** y la forma como **agregar** un nodo a la frontera permite definir diferentes **estrategias de búsqueda**.

buscarEn(Frontera) :-

➡ **seleccionar**(Nodo, Frontera, **FronteraReducida**),
generarVecinos(Nodo, **Vecinos**),

➡ **agregar**(**FronteraReducida**, **Vecinos**, **NuevaFrontera**),
buscarEn(**NuevaFrontera**).

Ejemplos:

Seleccionar	Agregar	Estrategia
El primero de "Frontera"	Al principio de Frontera	búsqueda en profundidad
El primero de "Frontera"	Al final de Frontera	búsqueda a lo ancho

Grafo que representa el espacio de estados

Búsqueda en profundidad - DFS (Depth-first search)

En **DFS (Depth-First Search)** la estrategia es retirar el primero de la lista que almacena la frontera y agregar los nuevos nodos generados al principio de la lista frontera (como una pila).

El gráfico representa al **árbol de búsqueda en profundidad**, que se va generando dinámicamente desde el estado inicial.

En verde están los nodos ya explorados y en rojo los nodos de la frontera. Observá que la frontera está formada por nodos "hermanos" de la rama que se está explorando.

Elijo	Agregar	Estrategia
El primero de "Frontera"	Al principio de Frontera	búsqueda en profundidad

Búsqueda en profundidad - DFS (Depth-first search)

En **DFS (Depth-First Search)** la estrategia es retirar el primero de la lista que almacena la frontera y agregar los nuevos nodos generados al principio de la lista frontera (como una pila).

`seleccionar( Nodo, [ Nodo | RestoLista] , RestoLista).`
`agregar( ViejaF, Vecinos, NuevaF ) :- append (Vecinos, ViejaF, NuevaF).`

buscarEn(Frontera) :-
➡ `seleccionar( Nodo, Frontera, FronteraReducida),`
 `generarVecinos( Nodo, Vecinos),`
➡ `agregar( FronteraReducida, Vecinos, NuevaFrontera),`
 `buscarEn(NuevaFrontera).`

El gráfico representa al **árbol de búsqueda en profundidad**, que se va generando dinámicamente desde el estado inicial. En verde están los nodos ya explorados y en rojo los nodos de la frontera. Observá que la frontera está formada por nodos “hermanos” de la rama que se está explorando y por ese motivo **la frontera crece de manera lineal**.

IA - Alejandro J. García

Búsqueda a lo ancho - BFS (Breadth-first search)

En **BFS (Breadth-First Search)** la estrategia es retirar el primero de la lista de la frontera y agregar los nuevos nodos generados al final de la frontera (como una cola).

El gráfico representa un **árbol de búsqueda a lo ancho**, que se va generando dinámicamente desde el estado inicial. En verde están los nodos ya explorados y en rojo los nodos de la frontera. Observá que en esta estrategia, la frontera está formada por los “hijos” del último nivel explorado.

Elijo	Agregar	Estrategia
El primero de “Frontera”	Al <u>principio</u> de Frontera	búsqueda en profundidad
El primero de “Frontera”	Al <u>final</u> de Frontera	búsqueda a lo ancho

IA - Alejandro J. García

Búsqueda a lo ancho - BFS (Breadth-first search)



En **BFS (Breadth-First Search)** la estrategia es retirar el primero de la lista de la frontera y agregar los nuevos nodos generados al final de la frontera (como una cola).

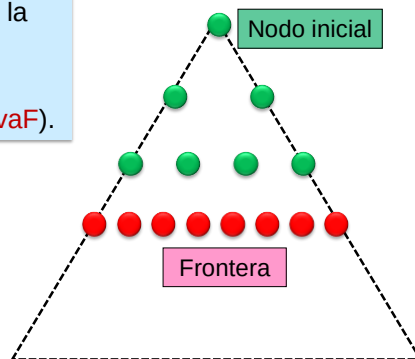
seleccionar(Nodo, [Nodo | RestoLista] , RestoLista).

agregar(ViejaF, Vecinos, NuevaF) :- append(ViejaF, Vecinos, NuevaF).

buscarEn(Frontera) :-

→ **seleccionar**(Nodo, Frontera, **FronteraReducida**),
generarVecinos(Nodo, Vecinos),

→ **agregar**(**FronteraReducida**, Vecinos, **NuevaFrontera**),
buscarEn(**NuevaFrontera**).

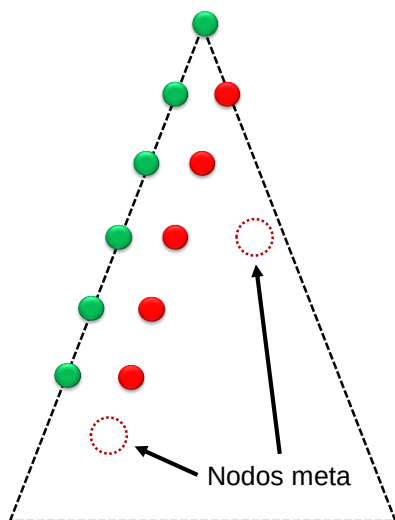


El gráfico representa un **árbol de búsqueda a lo ancho**, que se va generando dinámicamente desde el estado inicial. En verde están los nodos ya explorados y en rojo los nodos de la frontera. Observá que en esta estrategia, la frontera está formada por los “hijos” del último nivel explorado y **crece de manera exponencial**.

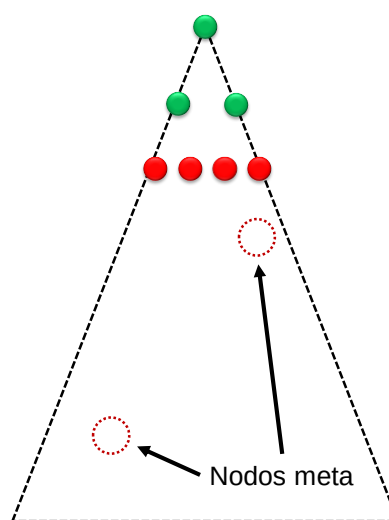


IA - Alejandro J. García

¿Qué meta encuentran primero?



¿Cuál meta encuentra DFS?



¿Cuál encuentra BFS?



IA - Alejandro J. García

Evaluación de los métodos de búsqueda



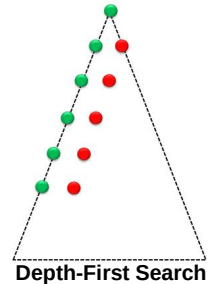
Complejidad: el algoritmo garantiza encontrar una solución si es que existe una.

Optimalidad: el algoritmo encuentra la solución óptima (el camino de menor costo).

Complejidad Temporal: número de operaciones aplicadas en la búsqueda.

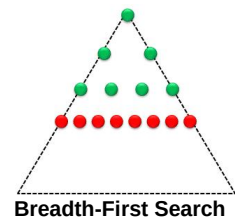
Complejidad Espacial: número de nodos almacenados durante la búsqueda.

Propiedad	Breadth-First Search	Depth-First Search
Completo	sí 😊	no 😞
Optimalidad	sí ⁽¹⁾ 😊	no 😞
Complejidad temporal	b^m 😞	b^m 😞
Complejidad espacial	b^m 😞	$b.m$ 😊



Depth-First Search

Puede haber una rama infinita



Breadth-First Search

b es el factor de ramificación y m es la profundidad de la meta

⁽¹⁾ Asumiendo que el costo de todos los operadores es el mismo.

IA - Alejandro J. García

BFS puede ser prohibitivo

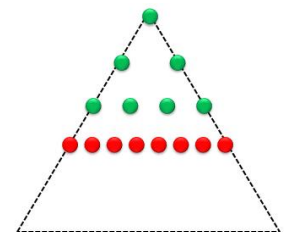


Considere por ejemplo: un factor de ramificación $b = 10$ (10 operadores)

Procesar 1 millón de nodos por segundo.

Cada nodo ocupa 1 Kbyte.

Profundidad (m)	Nodos frontera b^m	Tiempo	Memoria BFS
1	10	0.01 miliseg.	10 Kilobytes
4	10.000	11 miliseg.	10 Megabytes
6	10^6	11 segundos	1 Gigabyte
8	10^8	2 minutos	103 Gigabytes
10	10^{10}	3 horas	10 Terabytes
12	10^{12}	13 días	1 Petabyte
14	10^{14}	3,5 años	99 Petabytes
16	10^{16}	350 años	10 Exabytes



El uso de memoria en **BFS** puede ser prohibitivo.



IA - Alejandro J. García

DFS

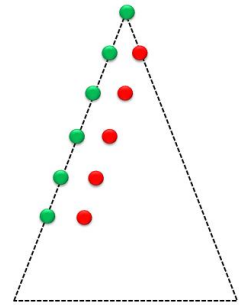


Considere por ejemplo: un factor de ramificación $b = 10$ (10 operadores)

Procesar 1 millón de nodos por segundo.

Cada nodo ocupa 1 Kbyte.

Profundidad (m)	Nodos frontera b.m	Tiempo	Memoria DFS
1	10	0.01 miliseg.	10 Kilobytes
4	40	11 miliseg.	40 Kilobytes
6	60	11 segundos	60 Kilobytes
8	80	2 minutos	80 Kilobytes
10	100	3 horas	100 Kilobytes
12	120	13 días	120 Kilobytes
14	140	3,5 años	140 Kilobytes
16	160	350 años	160 Kilobytes



El uso de **memoria en DFS es razonable** pero el **tiempo sigue siendo exponencial**.



IA - Alejandro J. García

Lowest-cost-first Search (LCFS) [PGM]



Observación: también llamado **Uniform-cost Search (UCS)** en el libro [RN] (ver sección 3.4.2)

BFS es óptimo si todos los arcos tienen el mismo costo. Con una simple modificación puede obtenerse una estrategia óptima para cualquier función de costo de paso (arco).

En un problema de búsqueda se puede tener una **función de costo de camino $g(n)$** que determina el costo para llegar del estado inicial a otro estado "n".

En lugar de seleccionar de la frontera el nodo de menor profundidad (como en BFS), hay que **elegir el nodo N con menor valor de $g(N)$** o costo de camino.

Debe modificarse "agregar/3" para que quede primero el nodo que tenga menor $g(N)$.

De esta manera **la frontera es una cola con prioridad**.

buscarEn(Frontera) :-

```

seleccionar( Nodo, Frontera, FronteraReducida),
generarVecinos( Nodo, Vecinos),
agregar( FronteraReducida, Vecinos, NuevaFrontera),
buscarEn(NuevaFrontera).

```



IA - Alejandro J. García

¿ Alguna conclusión hasta ahora ? ¿ Ideas ?



DFS (Búsqueda en Profundidad)

Apropiado cuando:

- almacenamiento es restringido
- hay muchas soluciones
- ... (ver [PMG] pág 128)

No apropiado cuando:

- es posible caer en caminos infinitos
- hay soluciones a "poca profundidad"

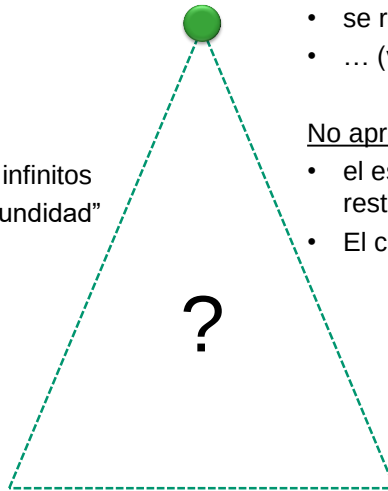
BFS (Búsqueda a lo ancho)

Apropiado cuando:

- almacenamiento no es problema
- se requiere la solución de menor profundidad
- ... (ver [PMG] pág 129)

No apropiado cuando:

- el espacio de almacenamiento es una restricción
- El costo de los operadores no es 1



¿ Nuevas ideas ?



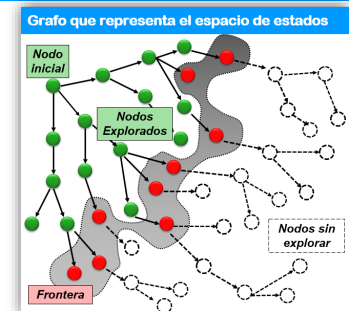
IA - Alejandro J. García

Resolución de Problemas utilizando búsqueda



Métodos de búsqueda no informados ("ciegos")

- búsqueda en profundidad (DFS)
- búsqueda a la ancho (BFS)
- Low cost-first (LCFS)
también llamado Uniform-cost Search (UCS)
- [opcional: no se evalúa en los exámenes]
Profundización iterativa (ID) [\[ver video\]](#)



Cada uno de los métodos detallados sigue una estrategia diferente de búsqueda, pero **todos recorren el espacio de búsqueda hasta que se "tropiezan" con la solución.**

Exploran el espacio de búsqueda "a ciegas" **sin usar ningún tipo de información** que "dirija" la búsqueda hacia nodos sin explorar que sean "más promisorios".

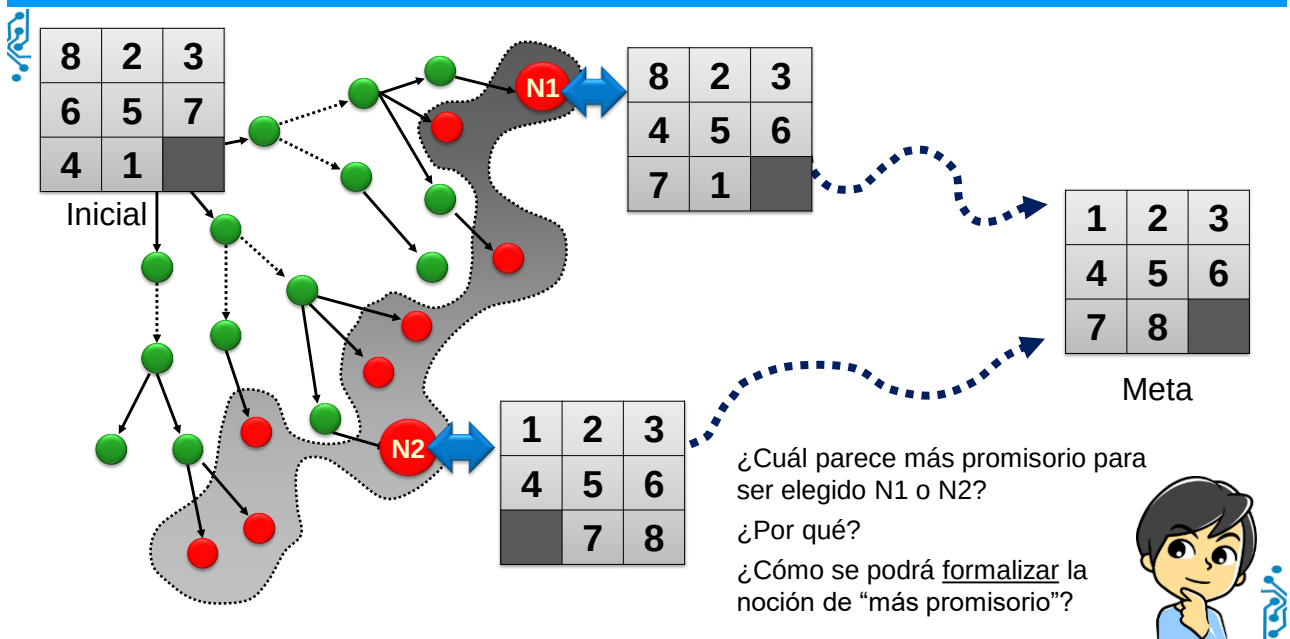
Se los llama estrategias de búsqueda ciega (*blind search*) o también estrategias no informadas (*uninformed search strategies*) en contraposición con las estrategias que sí usan información del problema para enfocar la búsqueda.



IA - Alejandro J. García

¿Puede hacerse una búsqueda informada?

¿Por qué no elegir el nodo más promisorio de la frontera?



IA - Alejandro J. García

¿Cómo formalizar el concepto de “nodo más promisorio”?

¿Cuál es el más promisorio?



N1:	<table><tr><td>1</td><td>2</td><td>3</td></tr><tr><td>4</td><td>8</td><td>5</td></tr><tr><td></td><td>7</td><td>6</td></tr></table>	1	2	3	4	8	5		7	6	N2:	<table><tr><td>1</td><td>2</td><td>3</td></tr><tr><td>4</td><td>5</td><td>6</td></tr><tr><td></td><td>7</td><td>8</td></tr></table>	1	2	3	4	5	6		7	8	N3:	<table><tr><td>8</td><td>2</td><td>3</td></tr><tr><td>4</td><td>5</td><td>6</td></tr><tr><td>7</td><td>1</td><td></td></tr></table>	8	2	3	4	5	6	7	1		N4:	<table><tr><td>8</td><td>2</td><td>3</td></tr><tr><td>6</td><td>5</td><td>7</td></tr><tr><td>4</td><td>1</td><td></td></tr></table>	8	2	3	6	5	7	4	1	
1	2	3																																									
4	8	5																																									
	7	6																																									
1	2	3																																									
4	5	6																																									
	7	8																																									
8	2	3																																									
4	5	6																																									
7	1																																										
8	2	3																																									
6	5	7																																									
4	1																																										

Considerá el **dominio de las 8-tejas** con la meta mostrada a la derecha.

Ordená los cuatros nodos de arriba del más promisorio al menos promisorio, justificando tu respuesta.

- ¿Podés especificar formalmente lo que hiciste?
- ¿Qué información usaste para determinar que un nodo es más promisorio que otro?
- Esta forma que planteaste, ¿te servirá para cualquier otro dominio?

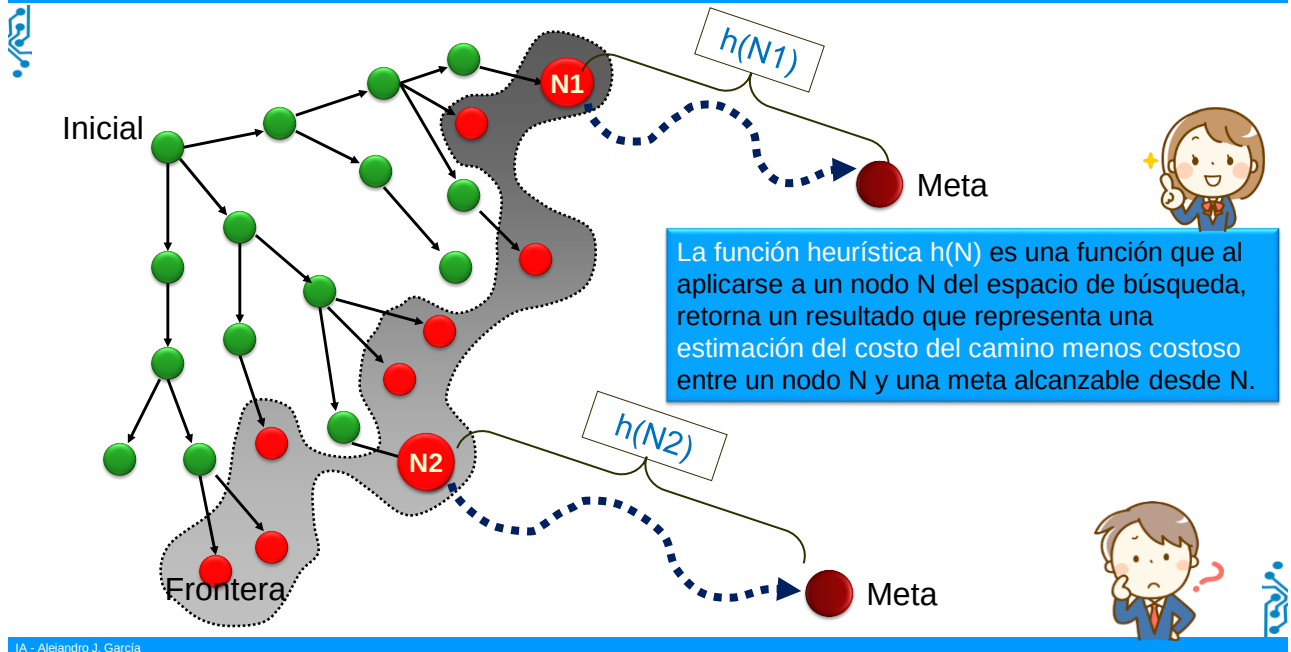
Meta:

1	2	3
4	5	6
7	8	



IA - Alejandro J. García

Función heurística



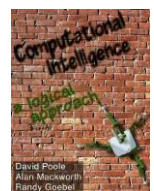
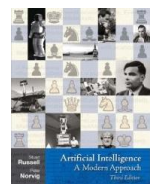
IA - Alejandro J. García

Bibliografía y material de referencia

Te sugiero utilizar este material de referencia:

- [RN] Russel, S., and Norvig, P. **Artificial Intelligence: A Modern Approach**. 3rd Ed.
Chapter 3: Problem solving by searching. (In which we see how an agent can find a sequence of actions that achieves its goals when no single action will do.)
- [PMG] Poole, D., Mackworth, A., and Goebel, R. **Computational Intelligence: A Logical Approach**.
Chapter 4: Searching (Many problem-solving tasks can be transformed to the problem of finding a path in a graph. [PMG, Ch. 4.2])

También podés ver estos videos que complementan al material de estudio que recopilé, redacté y organicé, y que está incluido en el resto del documento.



IA - Alejandro J. García

Esto no termina acá...



¡Muchas gracias por tu atención!

La búsqueda...

CONTINUARÁ

